

UT2 Estructura del Lenguaje JavaScript

Desarrollo Web en entorno Cliente (DEW)

Profesora: Yaiza Guanche Pérez
yguaper@gobiernodecanarias.org



Tabla de Contenidos

DEW

- 1. Fundamentos de JavaScript.
 - 1.1 Comentarios en el código.
 - 1.2 Constantes y variables
 - 1.3 Tipos de datos.
 - 1.31 Conversiones de tipos de datos.
 - 1.4 Operadores.
 - 1.4.1 Algunos operadores de interés.
 - 1.5 Condiciones y bucles.
 - 1.5.1 Estructuras de control.
 - 1.5.2 Bucles.

Objetivos

DEW

- Saber programar un script sencillo empleando la sintaxis básica del lenguaje.
- Saber usar variables en el código, así como realizar conversiones y operaciones básicas.
- Emplear bloques de código y estructuras repetitivas.

Fundamentos de JavaScript

DEW

1.1 Comentarios en el Código

- De una sola línea:

```
// Esto es un comentario
```

- De varias líneas:

```
/*
```

```
    Esto también es  
    un comentario
```

```
*/
```


Fundamentos de JavaScript

DEW

1.2 Constantes y variables

- Var declaramos una variable.

Es la forma más antigua de declarar variables.

Tiene ámbito de función, lo que significa que es accesible en toda la función donde se declara, incluso antes de su declaración, debido al hoisting (elevación).

Al ser JavaScript un lenguaje "débilmente tipado", no es necesario especificar el tipo de dato que vamos a almacenar en la variable, por tanto en la variable declarada podíamos almacenar un tipo de valor y luego otro tipo.

Fundamentos de JavaScript

DEW

1.2 Constantes y variables

Let

- Tiene ámbito de bloque, lo que significa que solo es accesible dentro del bloque donde se declara (por ejemplo, dentro de un if, for, etc.).
- No se puede acceder a una variable let antes de su declaración.
- JavaScript moderno ES6 (ECMAScript 2015), con él se consiguen muchas características nuevas que hicieron el lenguaje más potente, flexible y fácil de usar para los desarrolladores consiguiendo aplicaciones modernas y escalables en JavaScript.

1.2 Constantes y variables

`const`: palabra reservada para declarar constantes.

- También introducido en ES6.
- Tiene ámbito de bloque al igual que `let`.
- No se puede reasignar una vez que se le ha asignado un valor.
- Aunque no se puede reasignar, si la constante es un objeto o un array, sus propiedades o elementos internos sí pueden cambiar.

1.2 Constantes y variables

Nombres de variables:

- Comenzar por letra, símbolo, \$, _
- Contiene solo letras, números, \$ y _
- Son case sensitive.
- Identificadores únicos.
- No se pueden usar el nombre de palabras reservadas : break, continue, switch, if, for do, debugger, try ...catch, var, function, return.
- Se recomienda usar nombre descriptivos.

Fundamentos de JavaScript

DEW

1.2 Resumen.

- var: Variable con ámbito de función, permite reasignación.
- let: Variable con ámbito de bloque, permite reasignación.
- const: Constante con ámbito de bloque, no permite reasignación, pero las propiedades de objetos o arrays declarados con const pueden cambiar.

Fundamentos de JavaScript

DEW

- Sentencias:

Valores, operadores, expresiones, palabras clave y comentarios.

Es recomendable utilizar ; al final de cada sentencia.

Se pueden escribir varias sentencias en la misma línea separadas por ;

Los múltiples espacios en blanco se ignoran.

Se recomienda utilizar espacios entre operadores.

Bloques de código se agrupan entre llaves (funciones, bucles, condiciones,...)

Fundamentos de JavaScript

DEW

1.3 Tipos de datos:

- En Javascript existen dos grandes categorías:
- - primitivos: representan valores simples (number, string, boolean, undefined, null, symbol, bigint)
- - objetos: son colecciones de propiedades y métodos (objetos, arrays, funciones, date, map y set, RegExp)

Fundamentos de JavaScript

DEW

1.3 Resumen de los tipos de datos en JavaScript:

Tipos primitivos:	Tipos de referencia:
- number: Números, tanto enteros como decimales. - string: Texto o secuencias de caracteres. - boolean: Valores lógicos (true o false). - undefined: Valor por defecto de variables no inicializadas. - null: Representa ausencia de valor. - symbol: Identificadores únicos. - bigint: Números enteros grandes.	- object: Colección de propiedades y métodos. - array: Lista ordenada de elementos. - function: Bloque de código reutilizable. - date, map, set, regexp: Tipos de objetos más específicos.

Fundamentos de JavaScript

DEW

1.3 Tipos de datos:

Podemos usar **typeof** para saber el tipo de dato de una variable JavaScript.

- El tipo de datos de NaN es el número
- El tipo de datos de una array es objeto
- El tipo de datos de una date es objeto
- El tipo de datos de null es objeto
- El tipo de datos de un undefined variable es indefinido

No podemos usar **typeof** para objetos, pero si **instanceof** para verificar si un objeto es instancia de un constructor o clase.

Fundamentos de JavaScript

DEW

1.3 Tipos de datos:

1.3.1 Conversión tipo de datos:

- implícita: JavaScript realiza conversiones automáticas según el contexto.
- explícita cuando el programador lo cambia de forma intencionada, pero también se pueden usar funciones y métodos para convertir (Number(), String(), Boolean(), parseInt(), parseFloat()).
- Métodos internos: métodos como .toString() y .valueOf() son clave para convertir objetos a otros tipos.

Fundamentos de JavaScript

DEW

- 1.4 Operadores.
 - Operadores aritméticos: se utilizan para realizar operaciones matemáticas
 - + Suma y también se usa para concatenar.
 - Resta
 - * Multiplicación
 - / División
 - % Módulo
 - ++ Incremento (suma 1)
 - Decremento (resta 1)
 - ** Exponenciación

Fundamentos de JavaScript

DEW

1.4 Operadores.

- De Comparación: comparan dos valores y devuelven true o false.

== igual que (valor)

!= distinto (valor)

> mayor que

< menor que

>= mayor o igual que

<= menor o igual que

=== estrictamente igual que (compara tipos también)

!== estrictamente distinto que (compara tipos)

Fundamentos de JavaScript

DEW

1.4 Operadores.

- Lógicos:

&& Y lógico

|| O lógico

` OR

! Not (Negación)

Fundamentos de JavaScript

DEW

- 1.4 Operadores.

- Operadores de asignación: se utilizan para asignar valores a las variables

- = asigna un valor a una variable.

- += x Incrementa en x el valor de una variable.

- = x Decrementa en x el valor de una variable.

- *= x Multiplica en x el valor de una variable.

- /= x Divide en x el valor de una variable.

- %= Asignación de módulo ($x \% 2 \rightarrow x = x \% 2$)

- **= Asignación de exponenciación ($x ** 2 \rightarrow x = x ** 2$)

Fundamentos de JavaScript

DEW

- 1.4 Operadores.

- 1.4.1 Algunos operadores de interés.

- `+= x` Incrementa en x el valor de una variable.
 - `-= x` Decrementa en x el valor de una variable.
 - `*= x` Multiplica en x el valor de una variable.
 - `/= x` Divide en x el valor de una variable.
 - `++` Incrementa en 1 el valor de una variable
 - `--` Decrementa en 1 el valor de una variable.

Fundamentos de JavaScript

DEW

1.5 Condiciones y bucles.

1.5.1 Estructuras de control.

```
if (condición) {  
    // Código a ejecutar si se cumple la condición  
}  
else {  
    // Código a ejecutar si no se cumple la condición  
}
```

Podemos usar un operador ternario (?:) para asignaciones y condiciones sencillas.

Condición ? expresión_si_verdadero : expresión_si_falso;

¿Lo conocías?

Fundamentos de JavaScript

DEW

1.5 Condiciones y bucles.

1.5.1 Estructuras de control.

```
switch(expresión) {  
  case x:  
    // bloque de código  
  case y:  
    // bloque de código  
  default:  
    // bloque de código  
}
```

¿Usarías en tus códigos más if anidados o un switch? ¿Por qué?

Fundamentos de JavaScript

DEW

1.5 Condiciones y bucles.

1.5.1 Estructuras de control.

```
switch(expresión) {  
  case x:  
  case y:  
    // bloque de código  
  default:  
    // bloque de código  
}
```

Podemos agrupar los case x e y si van a tener el mismo código a ejecutar.

Fundamentos de JavaScript

DEW

1.5 Condiciones y bucles.

1.5.2 Bucles.

```
for(inicio;condición;incremento/decremento) {  
    // código a repetir mientras la condición sea cierta  
}
```

```
inicio;  
while (condición) {  
    //código a repetir mientras la condición sea cierta  
    incremento/decremento;  
}
```

```
inicio;  
do {  
    // código a repetir mientras la condición sea cierta  
    incremento/decremento;  
} while (condición);
```

¿Qué bucle es más eficiente?

Fundamentos de JavaScript

DEW

1.5 Resumen de bucles.

for: Se utiliza cuando conoces el número exacto de iteraciones.

while: Se utiliza cuando no sabes cuántas iteraciones, pero tienes una condición de parada.

do...while: Se asegura de ejecutar el bloque al menos una vez.

for...of: Para iterar sobre los valores de arrays o strings.

for...in: Para iterar sobre las claves de un objeto.

Fundamentos de JavaScript

DEW

1.5 Control de flujo en bucles.

break: sale del bucle inmediatamente.

continue: salta a la siguiente iteración, omitiendo el código restante del bloque.